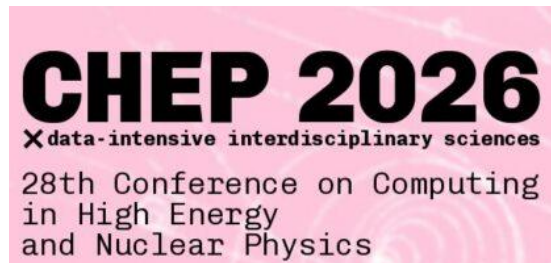


# Advanced Scheduling Strategies for Improved CPU Efficiency in CMS Offline Computing

A. Pérez-Calero Yzquierdo, Marco Mascheroni, Ralf Florian Von Cube, Hyun Woo Kim, Vaiva Zokaite, Luis Simas on behalf of the CMS Collaboration



# Outline of the talk

- The CMS Submission Infrastructure and its projection into HL-LHC
- CMS CPU efficiency recent results
- Scheduling in CMS SI: efficiency discussion and some results
- Payload CPU inefficiencies and pilot overloading strategy
- Conclusions

- The CMS Submission Infrastructure and its projection into HL-LHC

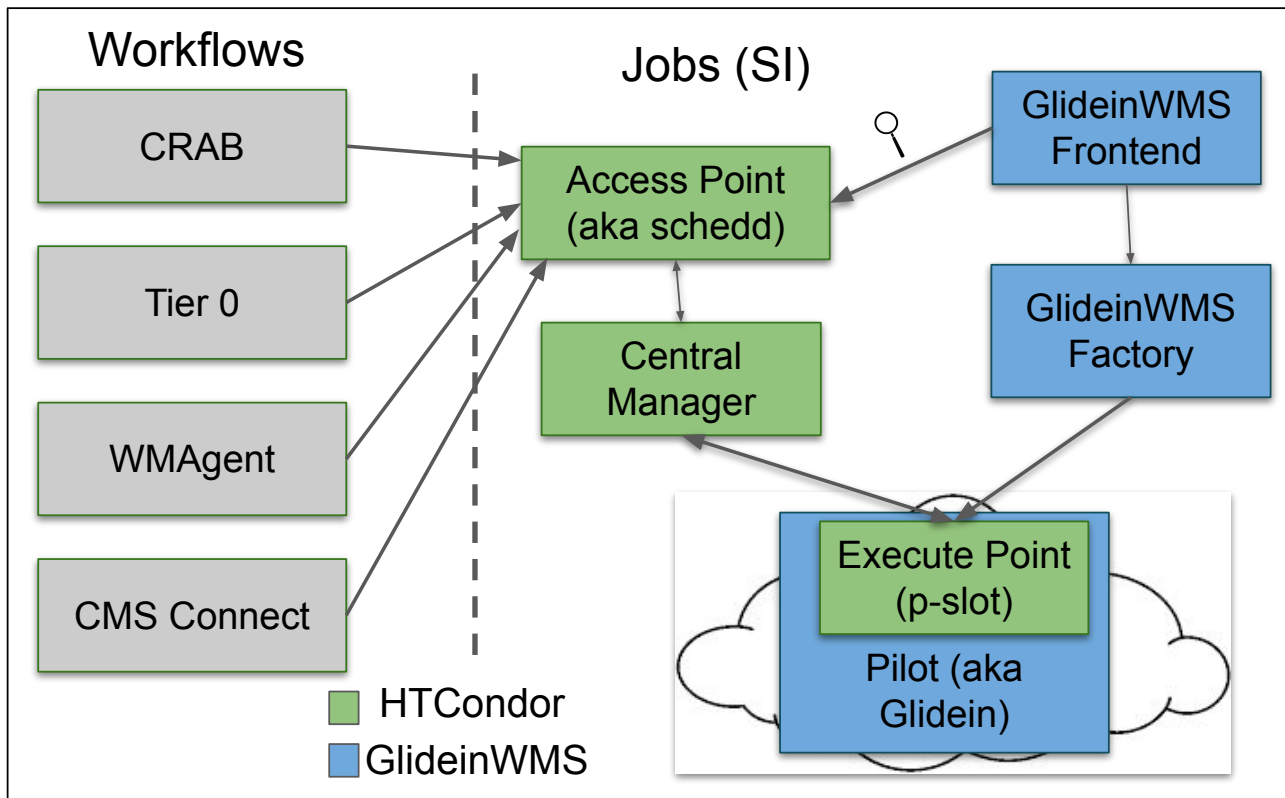
# The CMS Submission Infrastructure

## The CMS Sub. Infrastructure:

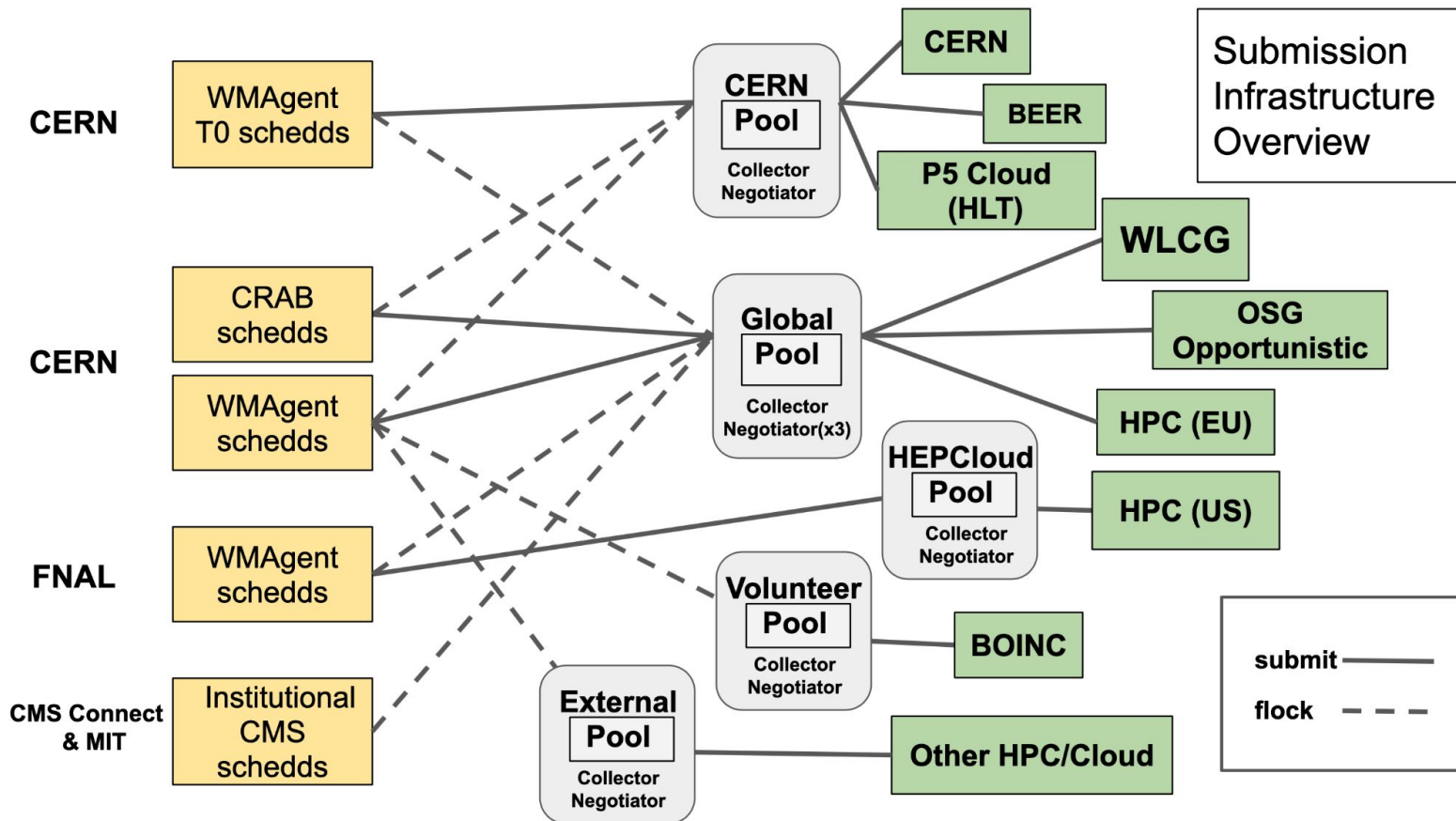
Acquire compute resources from providers (WLCG, HPC, Cloud) and make them available for CMS use in the form of a virtual unified pool

Enforce CMS priorities and quotas to the diverse CMS workflows and users

The CMS SI relies on a two-stage late-binding model supported by GlideinWMS and HTCondor

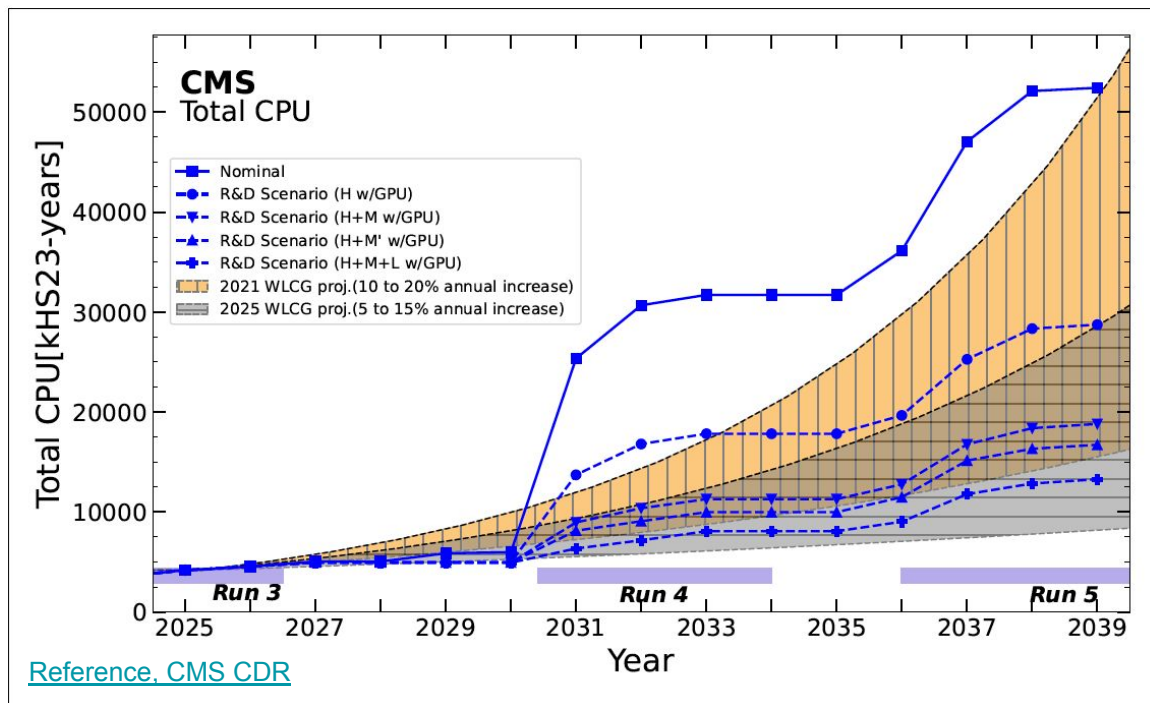


# The CMS Submission Infrastructure



# The HL-LHC Challenge for CMS Submission Infrastructure

The HL-LHC challenge for the CMS SI: continuous evolution enabling further **scalability** and **resource heterogeneity**, while maintaining the demonstrated stability, flexibility and **efficiency**



## The SI vision for HL-LHC:

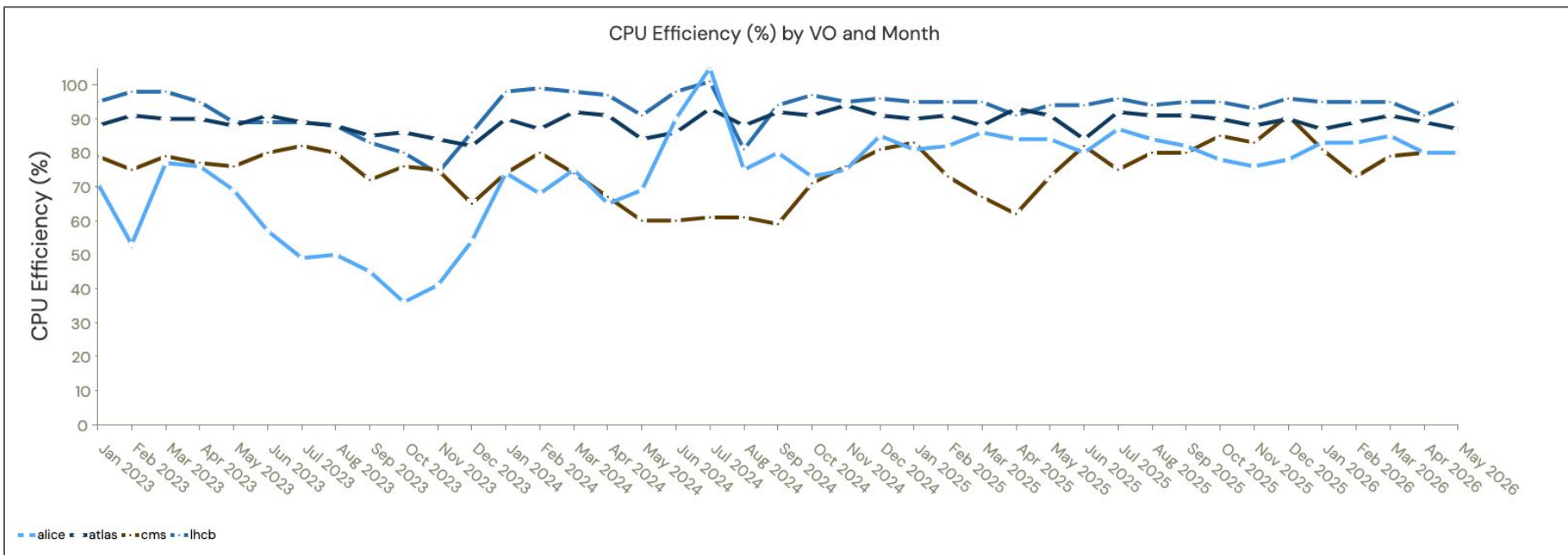
- Build from the current system that has served CMS effectively for more than a decade
- Expand and adapt to more and new resource types, diverse allocation policies and tools, etc

- CMS CPU efficiency recent results

# CMS CPU efficiency recent results

From the [EGI Accounting portal](#), monthly results are shown for CPU efficiency for LHC VOs at Tier-1 sites in recent years

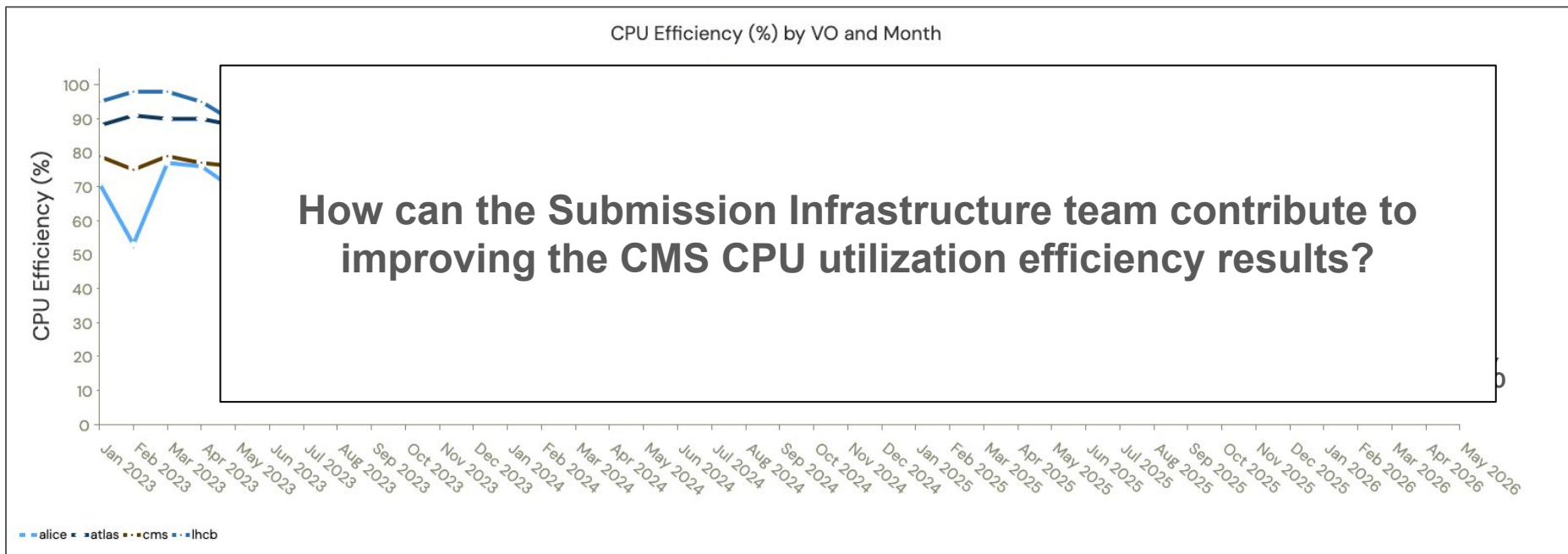
- **CMS result overall for the full period: 75%**
- CMS efforts are contributing to a **moderate but continuous trend towards higher efficiency results**



# CMS CPU efficiency recent results

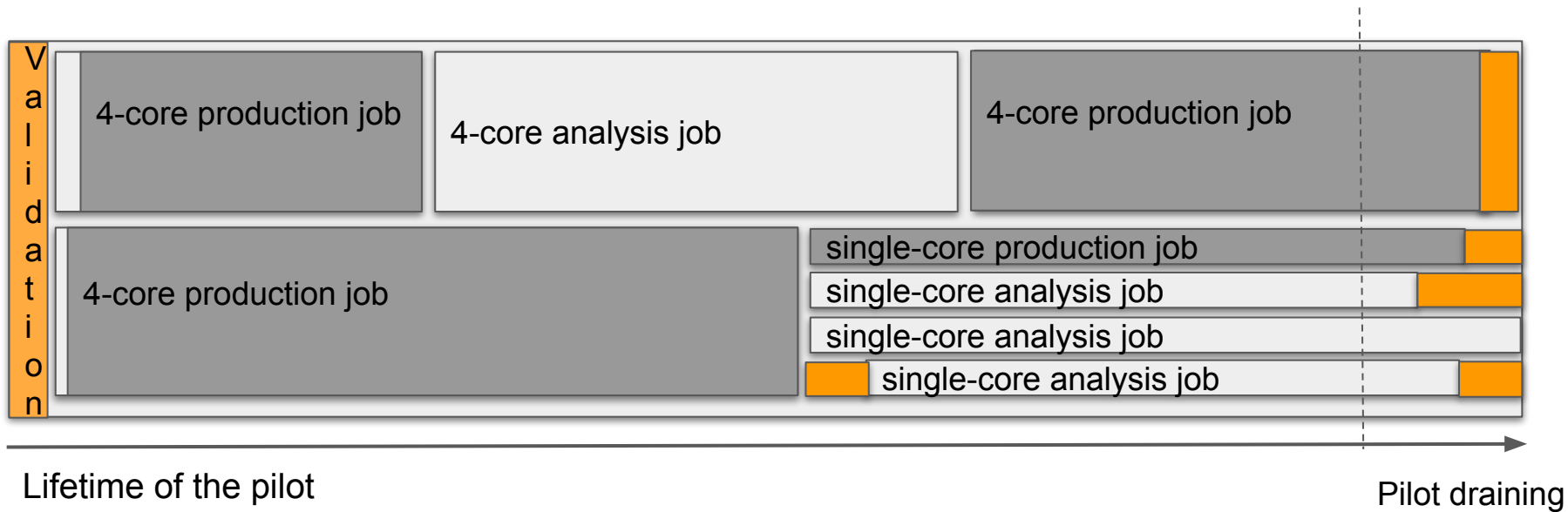
From the [EGI Accounting portal](#), monthly results are shown for CPU efficiency for LHC VOs at Tier-1 sites in recent years

- **CMS result overall for the full period: 75%**
- CMS efforts are contributing to a **moderate but continuous trend towards higher efficiency results**

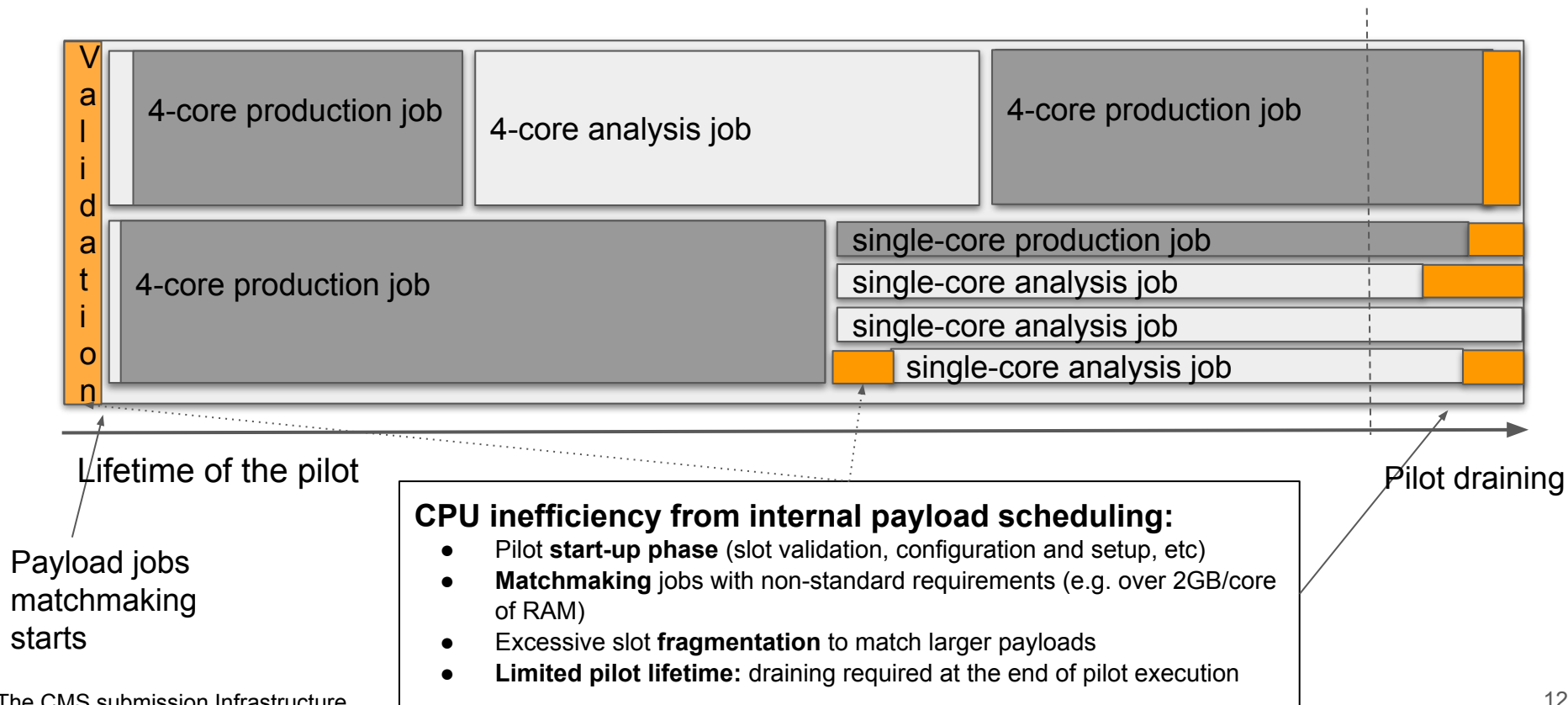


- Scheduling in CMS SI: efficiency discussion and some results

# CMS partitionable pilots executing multiple payload jobs



# CMS partitionable pilots executing multiple payload jobs



# The CMS SI in action at scale

## Acquiring resources:

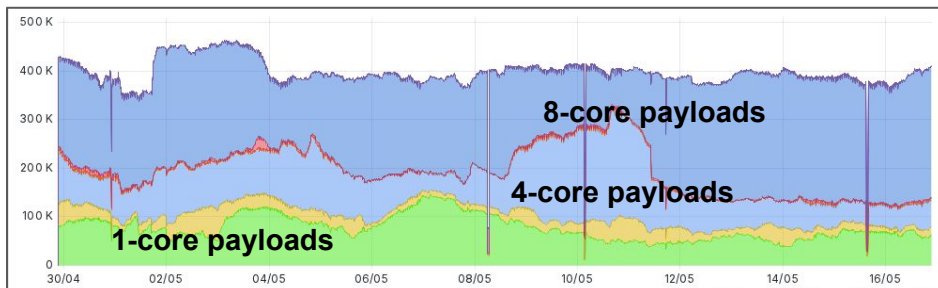
Mainly **16-core and 8-core pilots** from **WLCG sites**, but flexible enough to accept other core-counts, including e.g. **whole-node slots**, from WLCG sites or HPC.

No need for dedicated **high-memory slots**, average ~2.5 GB/core

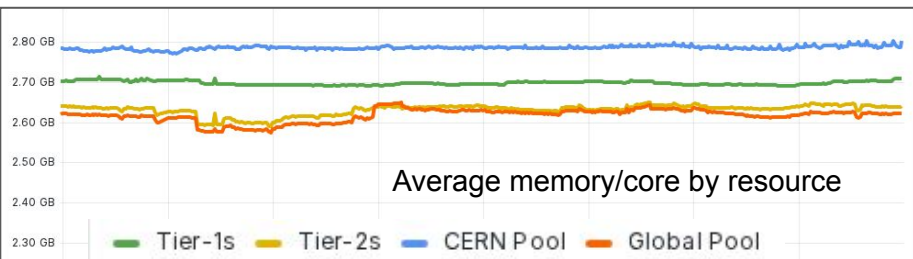
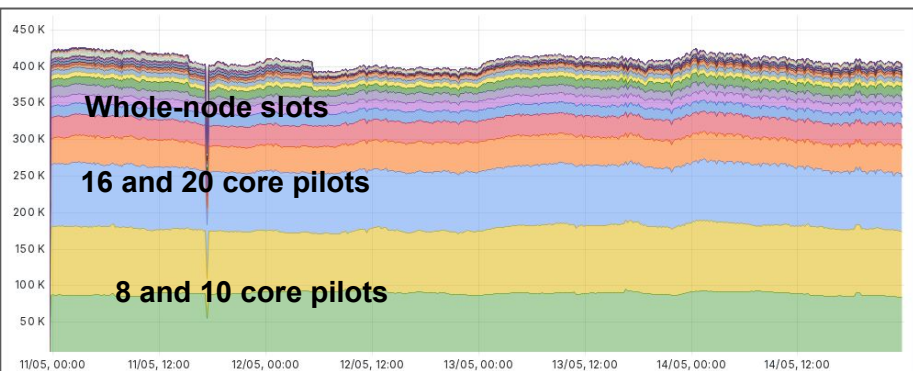
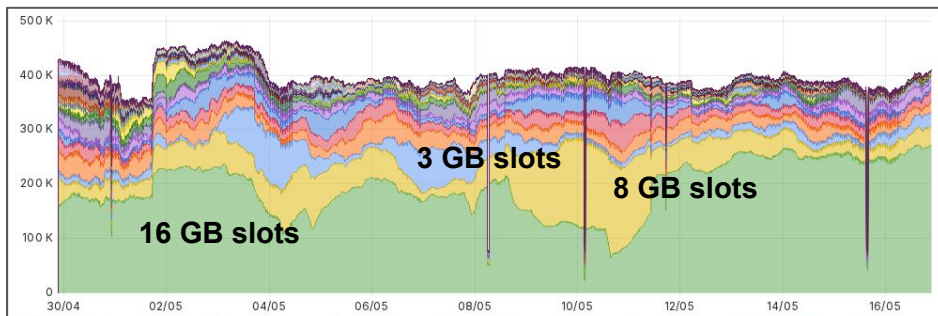
## Using resources:

Acquired slots are dynamically fragmented to match job resource requirements (CPU, memory, etc) and job types (**production vs analysis**)

## Dynamic slots by CPU cores



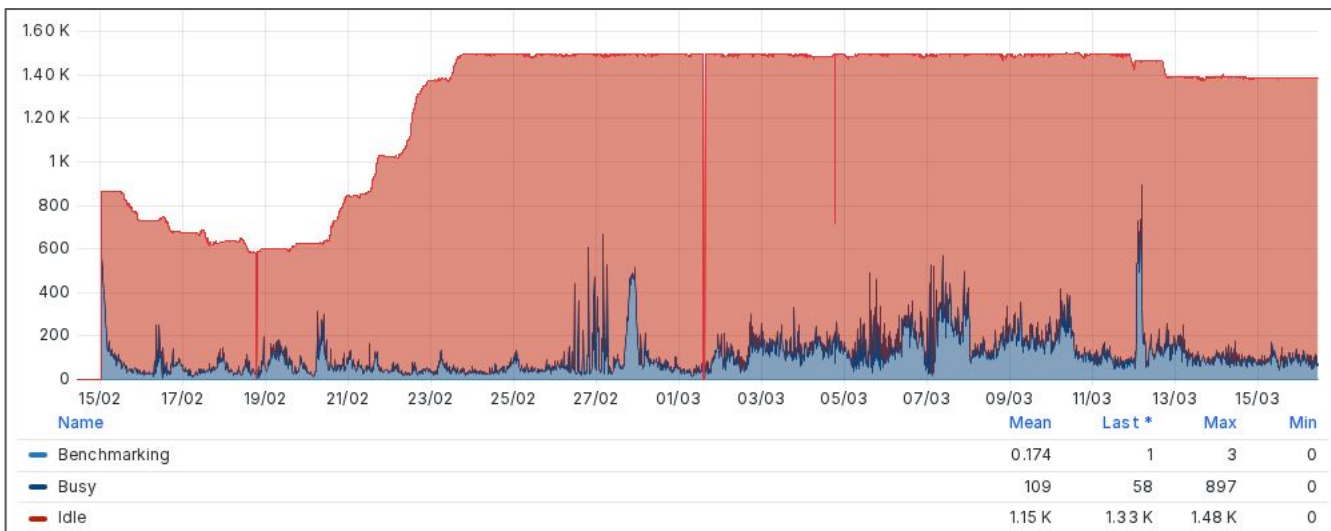
## Dynamic slots by memory



# IO-slots in SI

- Along with production and processing tasks, a number of short, **single-core, auxiliary payload jobs** need to be executed:
  - output file merge, log collect, etc.
- Bursts of these jobs produce **internal fragmentation of our slots**, impacting:
  - workload prioritization**: high-prio jobs (e. g. 8-core Tier0 prompt reconstruction) can't find enough large slots to run at required scale
  - efficiency**: additional draining is required for necessary defragmentation
- CMS SI has **introduced IO-slots**, as an additional slot per pilot matching exclusively such payloads
  - mainly used by merge jobs), the new slots **prevent excessive slot fragmentation**:

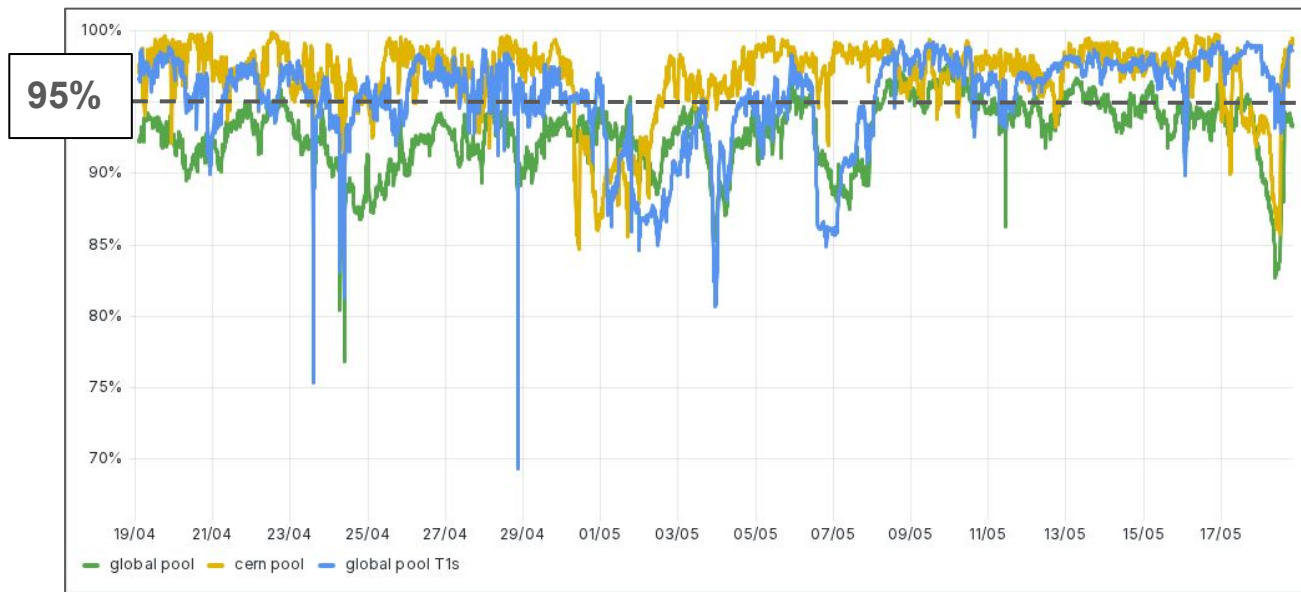
Example IO-slots in use (Tier-1s)



# Scheduling efficiency results

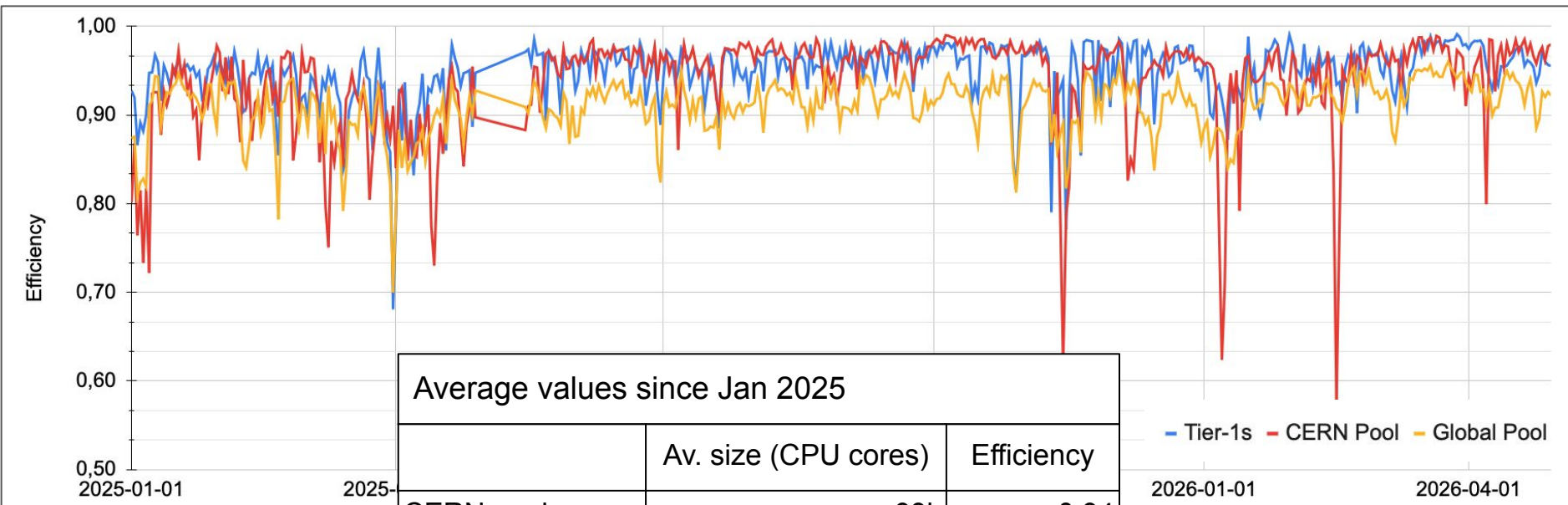
Considering all these causes affecting internal payload scheduling, CMS measures (and aims at maximizing) the slot utilization efficiency, which is typically **above 95% for CERN Pool and Tier-1s**

Scheduling efficiency for the last 30 days



# Scheduling efficiency results

**Slot utilization efficiency** in CMS SI since January 2025, for Global pool, CERN pool and Tier-1s, from daily averages of total resources busy and idle



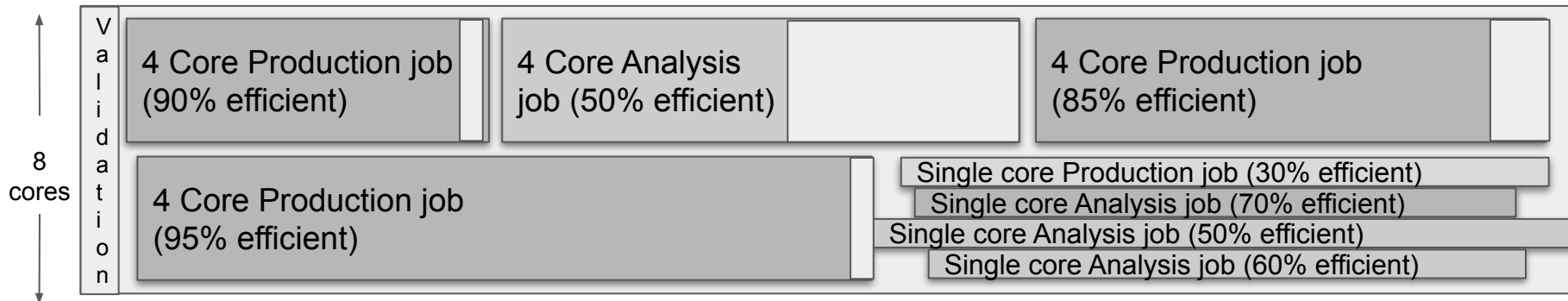
| Average values since Jan 2025 |                      |            |
|-------------------------------|----------------------|------------|
|                               | Av. size (CPU cores) | Efficiency |
| CERN pool                     | 99k                  | 0,94       |
| Tier-1s                       | 122k                 | 0,95       |
| Global pool                   | 354k                 | 0,91       |

- Payload CPU inefficiencies and pilot overloading strategy

# Payload CPU Inefficiency

- Payload jobs present several **sources of CPU inefficiency**
  - Bootstrapping and initializing
  - I/O-bound jobs, either **heavy I/O** jobs or jobs that employ **remote reads**
  - Suboptimal **user code** in analysis jobs
  - StepChain vs TaskChain workflows: Multiple **executables linked together as a single payload job**
    - Pro: less jobs to manage, reduce intermediate data storage and transfers, overall **much faster turnaround**.
    - Con: non-uniform resource needs leading to **inefficiencies**

Diverse reasons for inefficiencies, often difficult to directly reduce.



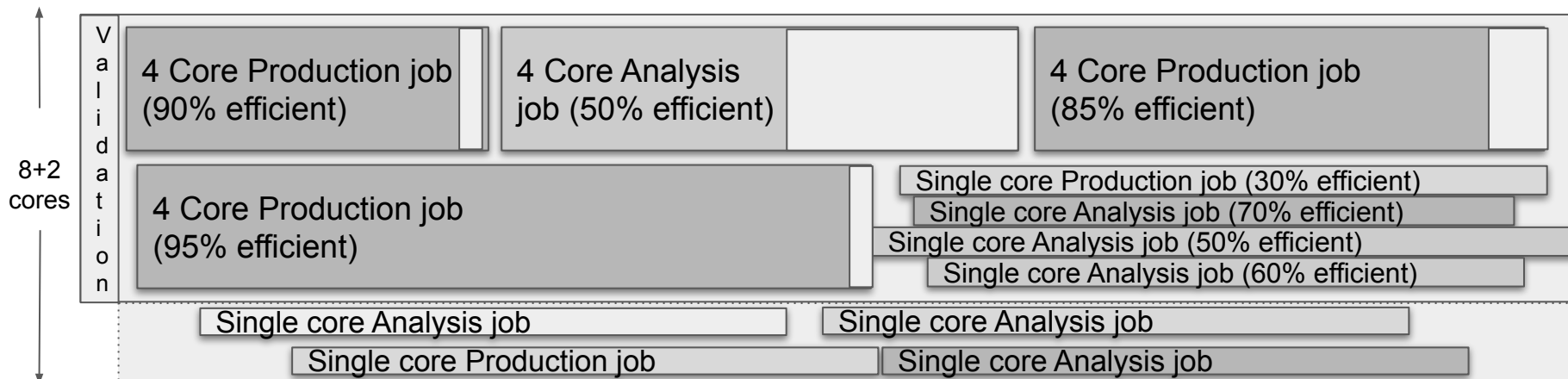
# Recover unused CPU cycles: overloading pilots

## Re-definition of the efficiency problem:

- **Improve CPU utilization efficiency by pushing more workload into the same pilot envelope**
  - Modify pilots so that **they accept more payload jobs into the same resources**
  - **Trivial to implement and test from CMS Submission Infrastructure side**

## Principle: recover unused CPU, **not** gain opportunistic cycles!

- **Moderate overloading:** adding **25%** extra CPU cores and memory to the nominal values of our pilots provides extra cores to match additional payload jobs

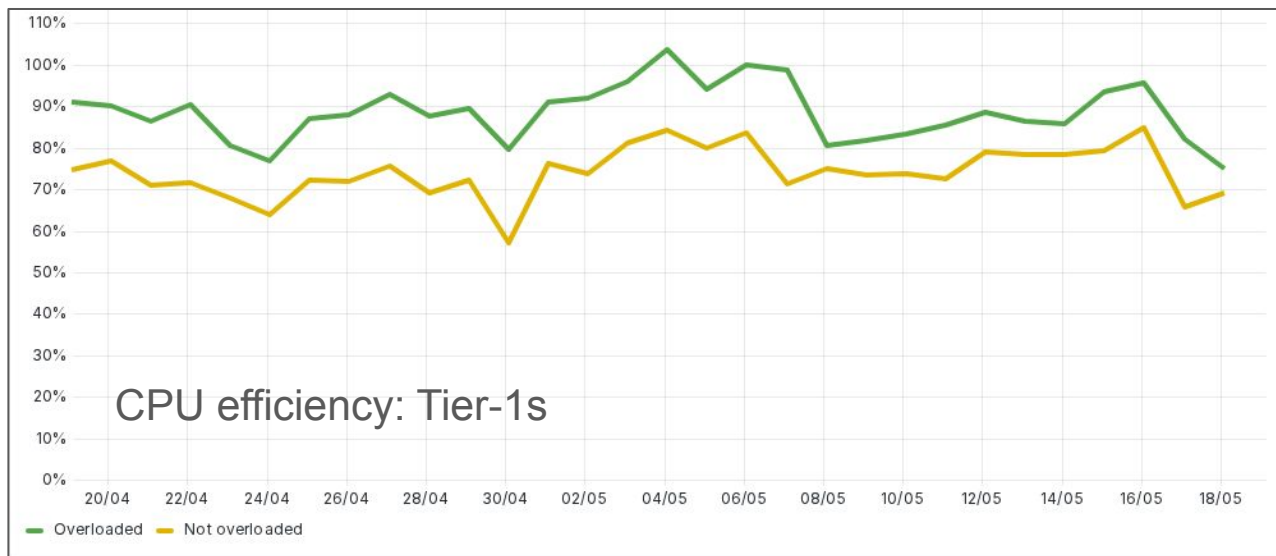


# Pilot overloading

- Introduced in 2023 as a test, based on manually **tuning some pilot factory entries, corresponding to specific sites' CEs**
- A precise **control mechanism now included in GlideinWMS**
  - Specify overloading percentage per pilot
  - For each CE, adjust the fraction of pilots using overloading
- **The strategy is already extended to about  $\frac{2}{3}$  of the Global Pool resources**
  - Continue with some pilots unchanged for reference
  - No point including some sites with pilots frequently not saturated by payloads
  - Some critical resources not overloaded
    - Some CERN pool slots, running critical Tier-0 tasks, present lower relative memory/core (e.g. offline use of Run2 HLT farm)

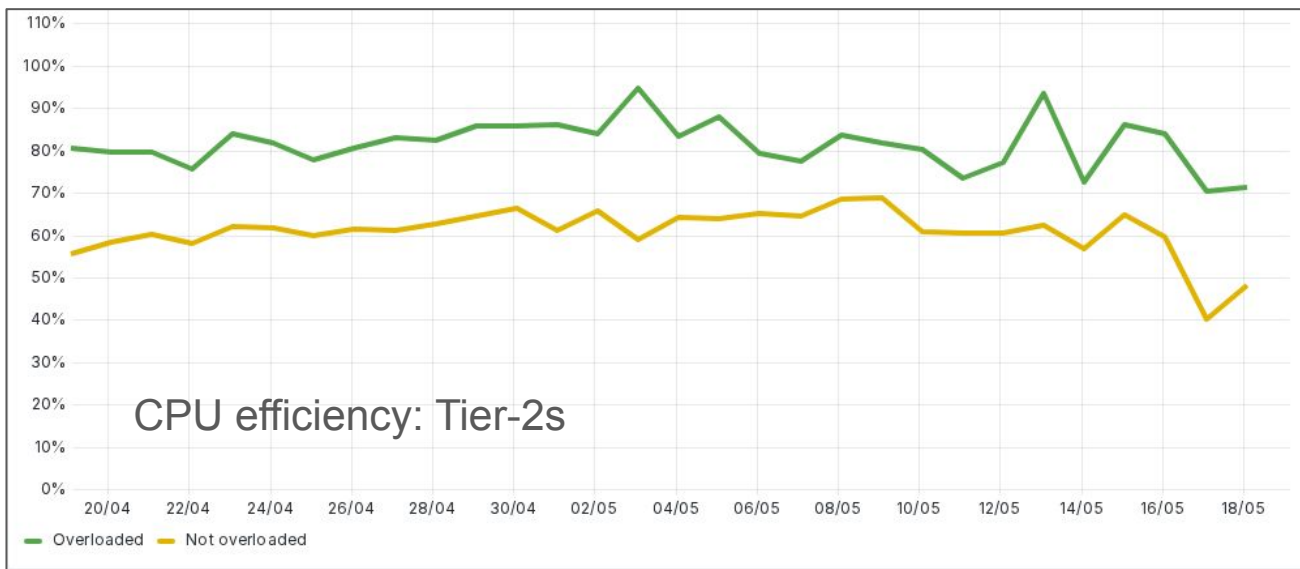
# Overloading results

- We can measure the **total CPU utilization for Sys+User processes** in a given GlideinWMS pilot, which can then be compared to **total Walltime x Ncores**
  - Allows us to compute the full **CPU efficiency (scheduling x payloads) per pilot**
  - Equivalent to the CPU efficiency metric observed by sites
- Deployed the monitoring to **continuously measure pilot performance**, comparing overloading on/off: **a significant and systematic improvement is observed**



# Overloading results

- We can measure the **total CPU utilization for Sys+User processes** in a given GlideinWMS pilot, which can then be compared to **total Walltime x Ncores**
  - Allows us to compute the full **CPU efficiency (scheduling x payloads) per pilot**
  - Equivalent to the CPU efficiency metric observed by sites
- Deployed the monitoring to **continuously measure pilot performance**, comparing overloading on/off: **a significant and systematic improvement is observed**



# Conclusions

# Conclusions

- CMS Submission Infrastructure **vision for HL-LHC** involves keeping the model we have and expanding it to ensure we can fully use more resources, in **scale** and **heterogeneity**, while maintaining stability, flexibility and **efficiency**
- CMS has dedicated a lot of **effort to improving the overall CPU utilization**, analyzing causes at both, the internal scheduling in pilots, and in the actual payload jobs
- Scheduling is very efficient already, at 95% at main sites, comparable to results local farms
- In order to compensate for the difficulty to reduce CPU inefficiency internal to CMS payload jobs, pilot overloading concept was introduced
- **Pilot overloading** strategy has now been **implemented** via new GlideinWMS tools, supported by new efficiency monitoring algorithms, revealing **a significant and systematic improvement across multiple sites**

# Acknowledgements

A. Pérez-Calero Yzquierdo's research supported by project PID2022-142604OB-C21, funded by



This work was partially supported by the US National Science Foundation under Grant No. 2121686 and under Grant No. 2030508

## Backup slides

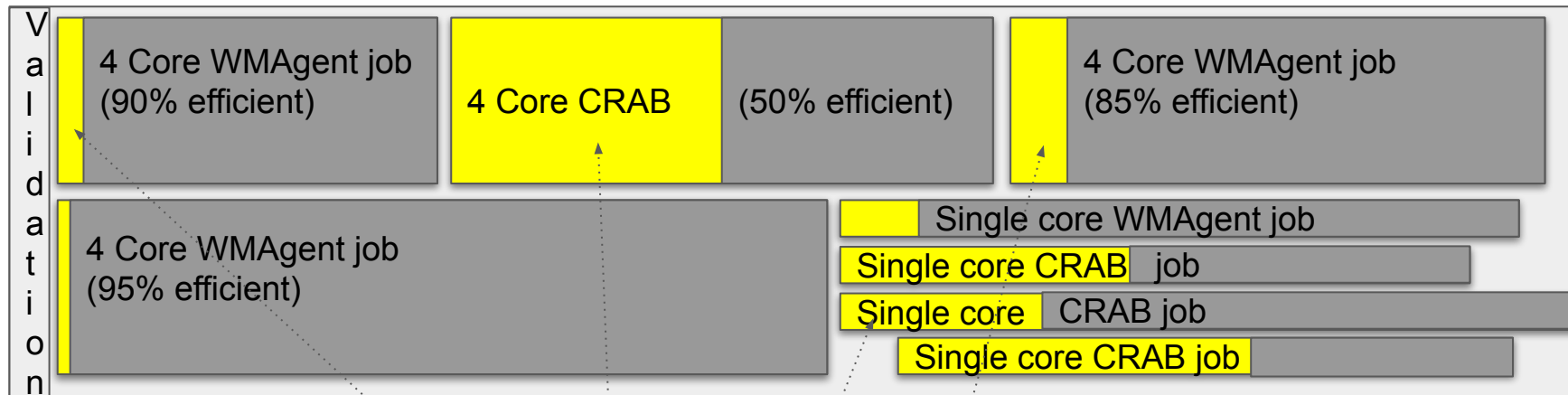
# Abstract

Efficient use of distributed computing resources is essential for sustaining the growing processing demands of the CMS experiment. Building on our previous work to assess and minimize unused CPU cycles, new advances in scheduling strategies that further improve resource utilization are being developed for the CMS Global Pool.

The CMS Submission Infrastructure team is deploying enhanced scheduling policies for our late-binding multicore pilot-job model, which is employed to aggregate and exploit large amounts of Grid and HPC compute resources. These policies include mechanisms for dynamically placing additional user payloads on pilot resources to maximize overall processing throughput, as well as the introduction of dedicated I/O-bound slots for jobs that are limited by data-access latency rather than CPU availability. Together, these strategies allow pilots to better balance job types and to optimize the utilization of compute capacity without affecting job success rates or remote sites stability.

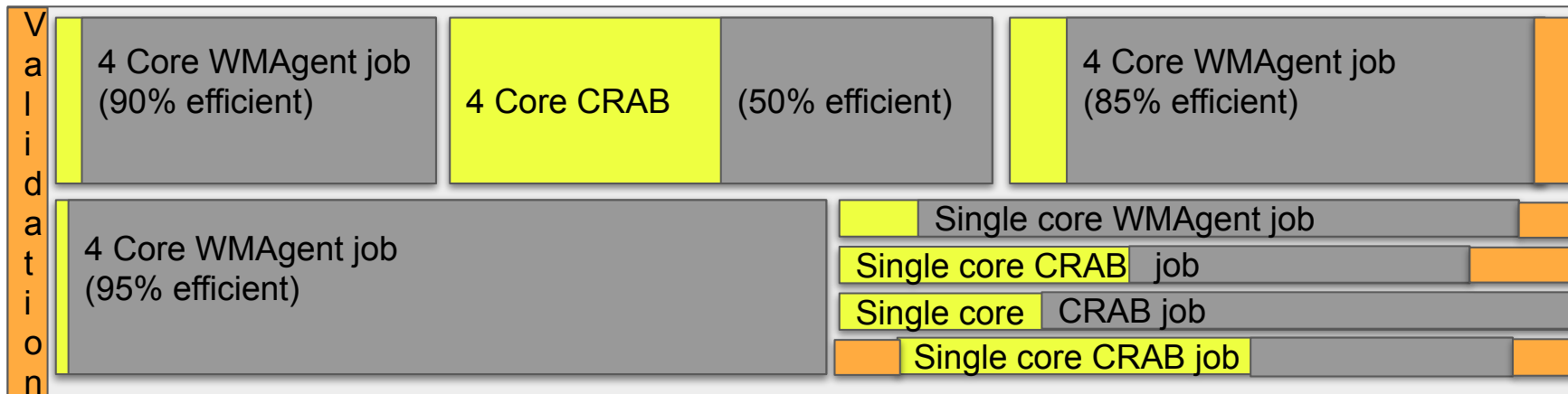
Results demonstrating consistent CPU efficiency gains across multiple resource providers will be presented, along with an analysis of their impact on workflow throughput. These developments highlight the effectiveness of advanced job-placement and scheduling techniques inside CMS pilots and represent an important step toward increasing the overall efficiency of the CMS Submission Infrastructure.

# A CMS 8-core 48h pilot job executing multiple payloads



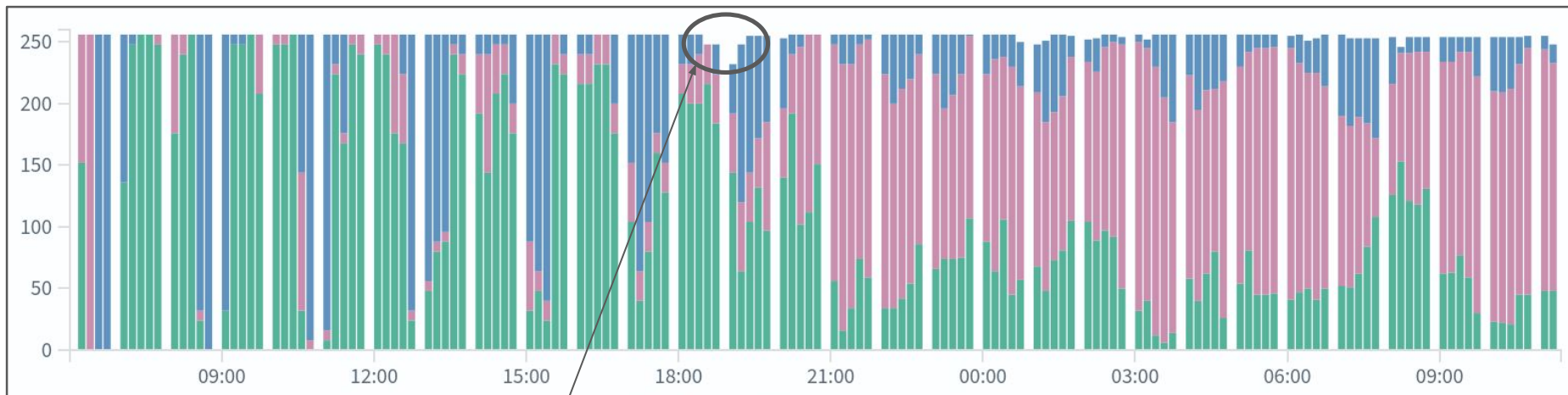
**Payload Inefficiencies**

# A CMS 8-core 48h pilot job executing multiple payloads



- **Efficiency results** observed and reported by our sites to the **EGI accounting portal** include **scheduling AND payload** Inefficiencies
- They can be factorized and measured independently

Example of a whole-node pilot (256 CPU cores) executing multiple payload jobs, showing full CPU use for its total runtime



Scheduling inefficiencies are minimal

Payload CPU  
efficiencies

